Indian Institute of Technology Dharwad

CS201L: Artificial Intelligence Laboratory

Lab 12: Clustering - K-Means and K-Medoid

Problem Statement

You are given a dataset file `tsne_2d_features_6_languages.csv` containing 2-dimensional t-SNE features extracted from speech audio embeddings. The dataset consists of 2500 audio samples from six different languages (English, Hindi, Kannada, Marathi, Bengali, Manipuri).

Columns 1 and 2 (`tsne_1` and `tsne_2`) represent the 2D feature coordinates. Column 3 (`language`) is the ground truth class label associated with each audio sample. The task here is to partition (cluster) the 2D feature data using K-Means clustering and K-Medoid clustering, and evaluate the quality of the clusters. While performing the clustering, ignore the third column (`language`) of the data. Use the class information in the third column *only* for computing the evaluation metrics.

1. Data Loading and Exploration

(a) Load the audio features dataset from the CSV file.

(b) Separate the features (`tsne_1`, `tsne_2`) and the true labels (`language`).

(c) Visualize the dataset using a scatter plot shown in figure 1, assigning a distinct color to each of the 6 languages.

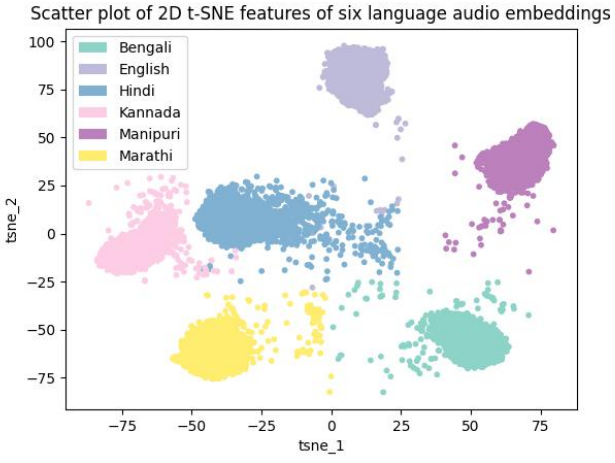

Figure 1: Scatter plot of 2D t-NSE feature dataset with language label

2. K-Means Clustering

Perform the K-means clustering and all the steps below for the number of clusters (K) as 2, 3, 4, 5, 6, 7, 8, 9, and 10. Record the distortion measure (inertia) and performance metric (purity score, Normalized Mutual Information (NMI), and Silhouette score) for each of the K values.

(a) Apply K-Means clustering on the 2D feature dataset.

(b) Assign cluster labels to each data point using `KMeans.labels_`.

(c) Visualize the clustered data using a scatter plot, coloring the points based on their assigned cluster labels. Also, show the centre of each cluster. (Hint: use `KMeans.cluster_centers_`)

(d) Compute the distortion measure (inertia) of each K. (Hint: use `KMeans.inertia_`)

(e) Compute the **purity score** after examples are assigned to clusters for each K. Write a program to compute the **purity score**.

(f) Compute the **NMI** score for each K.

(g) Compute the **Silhouette score** to measure how similar each sample is to its own cluster compared to other clusters for each K.

(h) Show all the computed performance metrics in a table form for all K values.

(i) Give the plot of K vs distortion measure. Find the heuristic optimum number of clusters using the elbow method for K-means clustering.

3. K-Medoid Clustering

Perform the K-Medoid clustering and all the steps below for the number of clusters (K) as 2, 3, 4, 5, 6, 7, 8, 9, and 10. Record the distortion measure (inertia) and performance metric (purity score, Normalized Mutual Information (NMI), and Silhouette score) for each of the K values.

(a) Apply K-Medoid clustering on the 2D feature dataset.

(b) Assign cluster labels to each data point using `KMedoids.labels_`.

(c) Visualize the clustered data using a scatter plot, coloring the points based on their assigned cluster labels. Also, show the centre of each cluster. (Hint: use `KMedoids.cluster_centers_`)

(d) Compute the distortion measure (inertia) for each K. (Hint: use `KMedoids.inertia_`)

(e) Compute the **purity score** after examples are assigned to clusters for each K. Write a program to compute the **purity score**.

(f) Compute the **NMI** score for each K.

(g) Compute the **Silhouette score** to measure how similar each sample is to its own cluster compared to other clusters for each K.

(h) Show all the computed performance metrics in a table form for all K values.

(i) Give the plot of K vs distortion measure. Find the heuristic optimum number of clusters using the elbow method for K-Medoid clustering.

Note

Please upload the completed Jupyter Notebook. Copy all Jupyter Notebooks and data files in a folder and rename the folder as your `rollnumber_Lab12`, then create the zip file with the name `rollnumber_Lab12.zip`. Finally, upload the zip file to Moodle for evaluation.

Functions / Code Snippets

```
# Code snippet for k mean clustering.
import pandas as pd
from sklearn.cluster import KMeans # Apply K-Means clustering
from sklearn_extra.cluster import KMedoids # K-Medoids clustering algorithm
from sklearn.metrics import normalized_mutual_info_score, silhouette_score #
↪  Compute NMI and Silhouette Score
import matplotlib.pyplot as plt # Visualization of clusters

# 1. Load data
tsne_df = pd.read_csv("tsne_2d_features_6_languages.csv")

# Apply K-Means Clustering
kmeans = KMeans(n_clusters=10, random_state=42)
kmeans.fit(tsne_df[['tsne_1', 'tsne_2']])
labels = kmeans.labels_

# Apply K-Medoid Clustering
kmedoids = KMedoids(n_clusters=10, random_state=42)
kmedoids.fit(tsne_df[['tsne_1', 'tsne_2']])
labels = kmedoids.labels_

# To compute distortion measure
inertia1 = kmeans.inertia_
inertia2 = kmedoids.inertia_

# To compute NMI
nmi = normalized_mutual_info_score(true_labels, cluster_labels)

# To compute Silhouette Score
features = tsne_df[['tsne_1', 'tsne_2']]
sil_score = silhouette_score(features, cluster_labels)

# To compute the purity score, follow the definition in the slide.
```

References

- Scikit-Learn K-Means documentation: `https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html`
- Scikit-Learn K-Medoid documentation: `https://scikit-learn-extra.readthedocs.io/en/stable/generated/sklearn_extra.cluster.KMedoids.html`